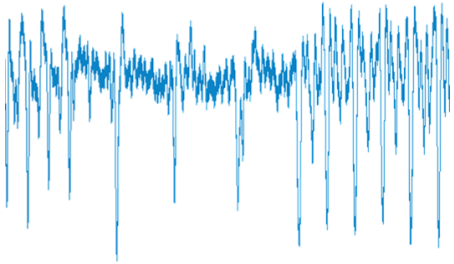
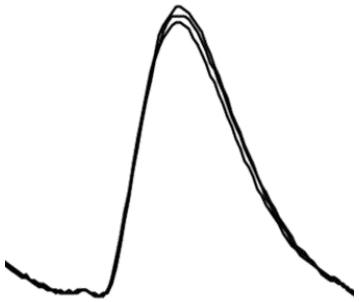
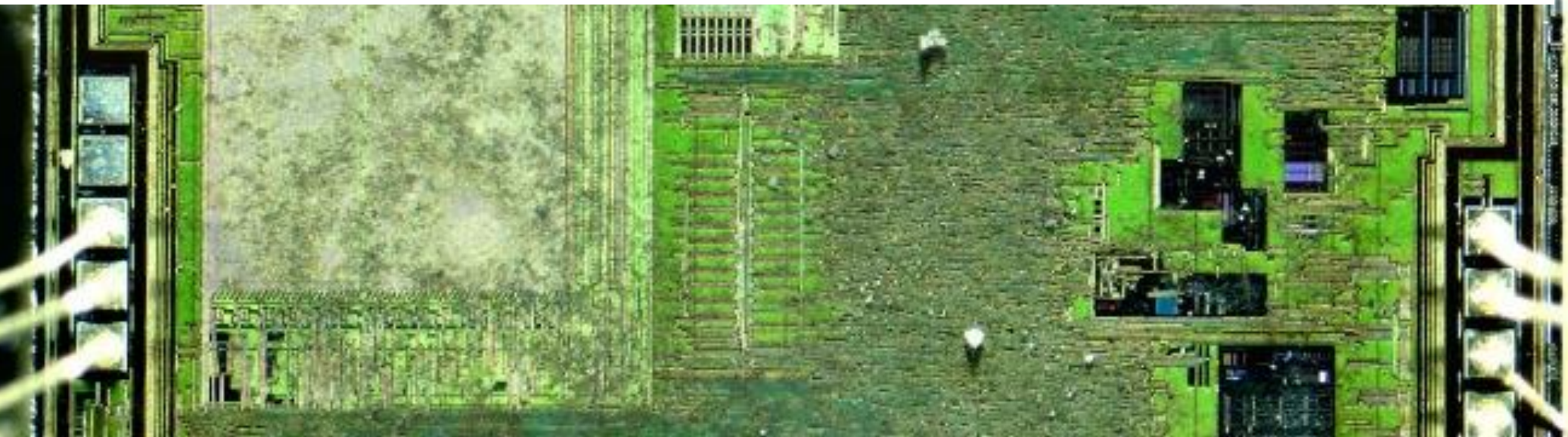
Code:

Side-channel attacks 2

David Oswald







Goals of this lecture

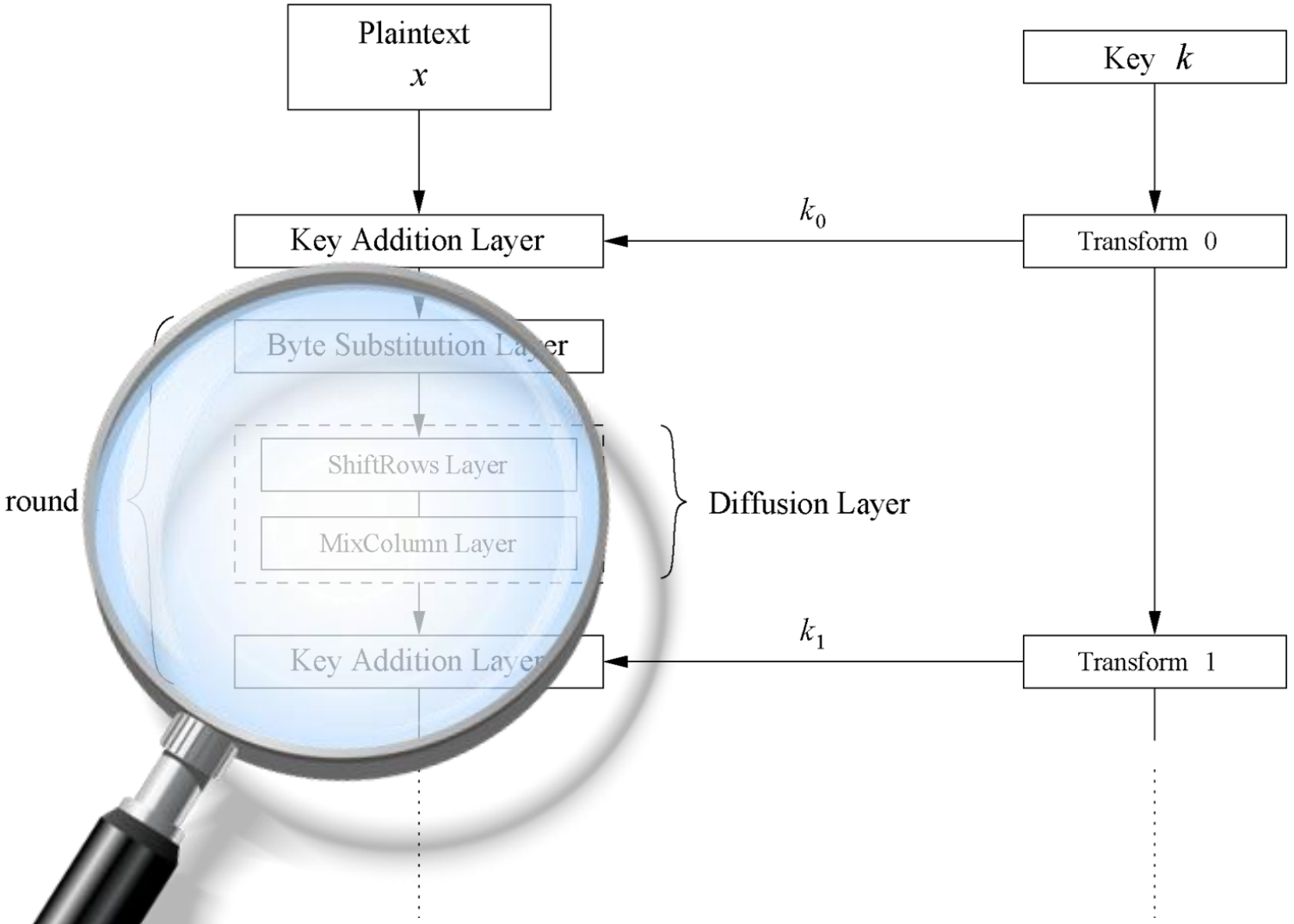
After this lecture, you will …

- Understand how power and timing side-channel attacks can break AES
- Know why it is a bad idea to implement AES by specification
- Be prepared for the next lectures

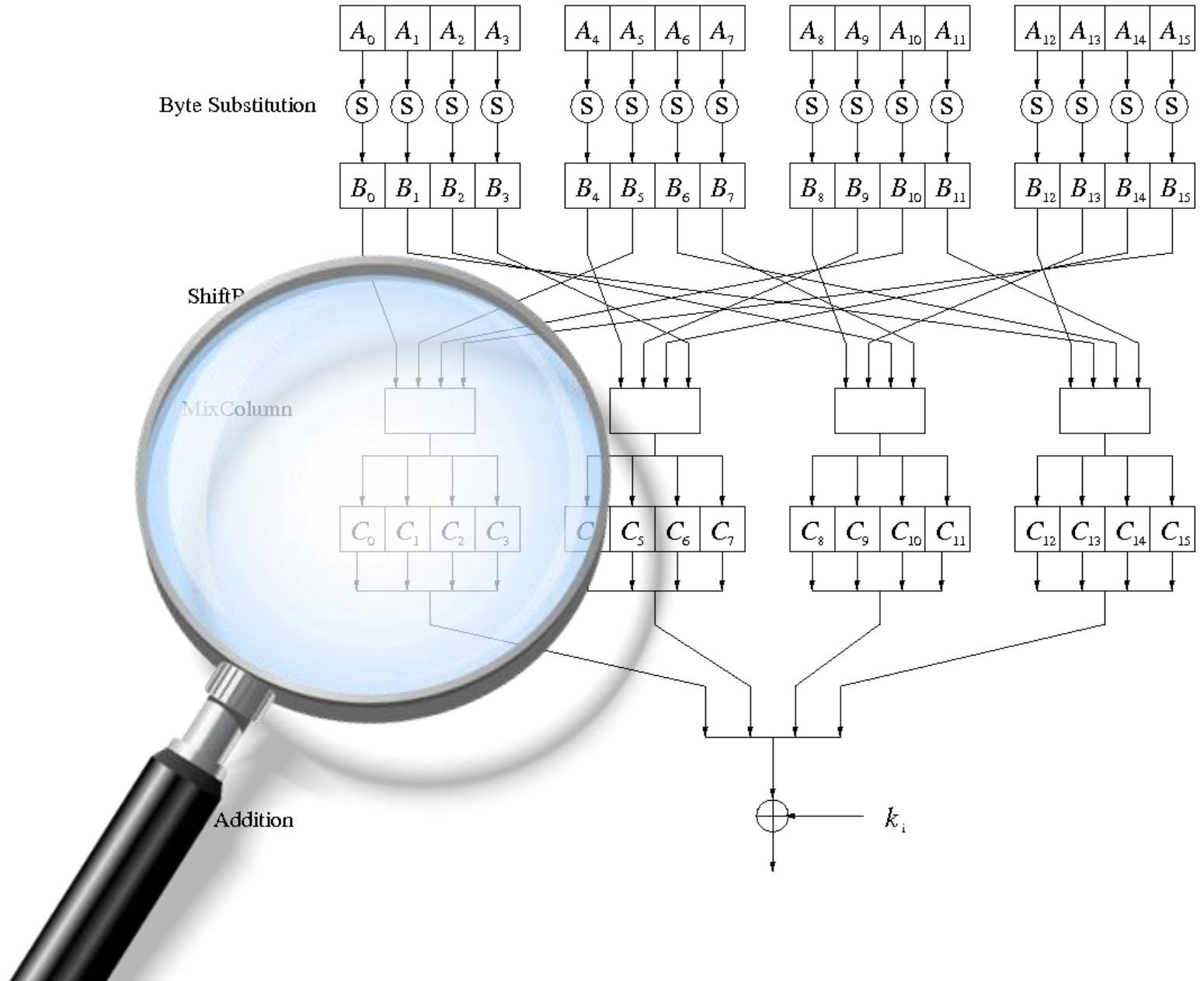
AES – a brief recap

- Actually called Rijndael, became the Advanced Encryption Standard in 2001 (https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf)
- Open competition run by NIST
- We'll look at the 128-bit key version with 10 rounds

AES: overview



AES: Round function



AES: MixColumns

Linear transform: 4 input bytes generate 4 output bytes

$$\begin{pmatrix} C_0 \\ C_1 \\ C_2 \\ C_3 \end{pmatrix} = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix} \cdot \begin{pmatrix} B_0 \\ B_5 \\ B_{10} \\ B_{15} \end{pmatrix}$$

e.g., $C_0 = \underbrace{02\ B_0}_{0x1B} + 03\ B_5 + B_{10} + B_{15}$

$02\ B_0$ really is $x \cdot B_0 \bmod x^8 + x^4 + x^3 + x + 1$

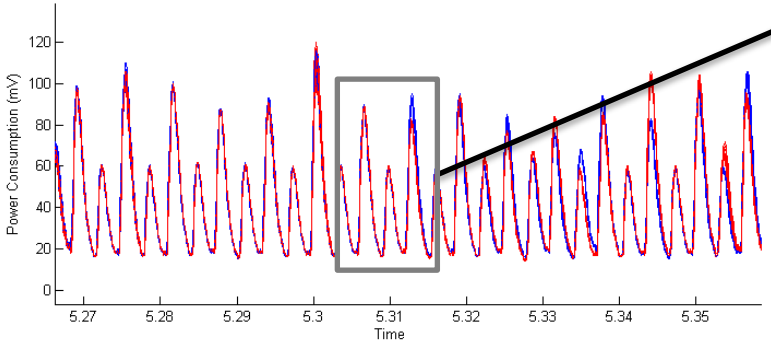
This is also known as xtime()…

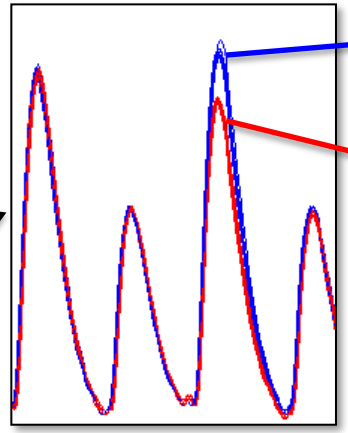
The xtime() operation

```
uint8_t xtime(uint8_t B)
{
    uint8_t tmp = B << 1;
    if(B & 0x80 == 0x80)
    {
        tmp ^= 0x1B;
    }
    return tmp;
}
```

AES: SPA-like attack on xtime()

```
uint8_t tmp = B << 1;
if(MSBit(B) == 1)
    tmp ^= 0x1B;
```





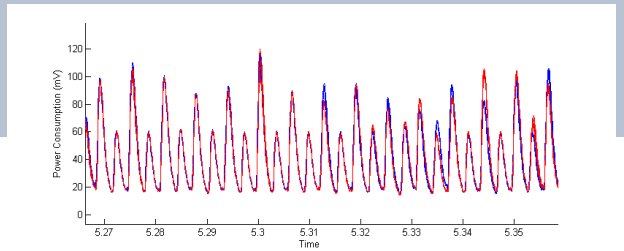
XOR executed:
MSBit(B) == 1

XOR not executed:
MSBit(B) == 0

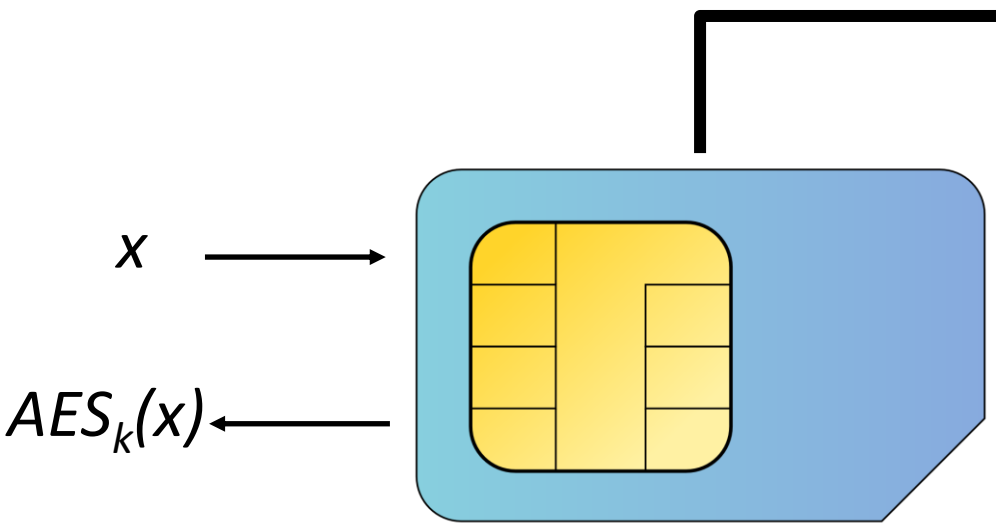
Attack outline

1. Observe program flow in power trace
2. Learn MSBit of B_0
3. Recovers key byte
4. Repeat for each byte separately

Attack setup



Power trace p(t)



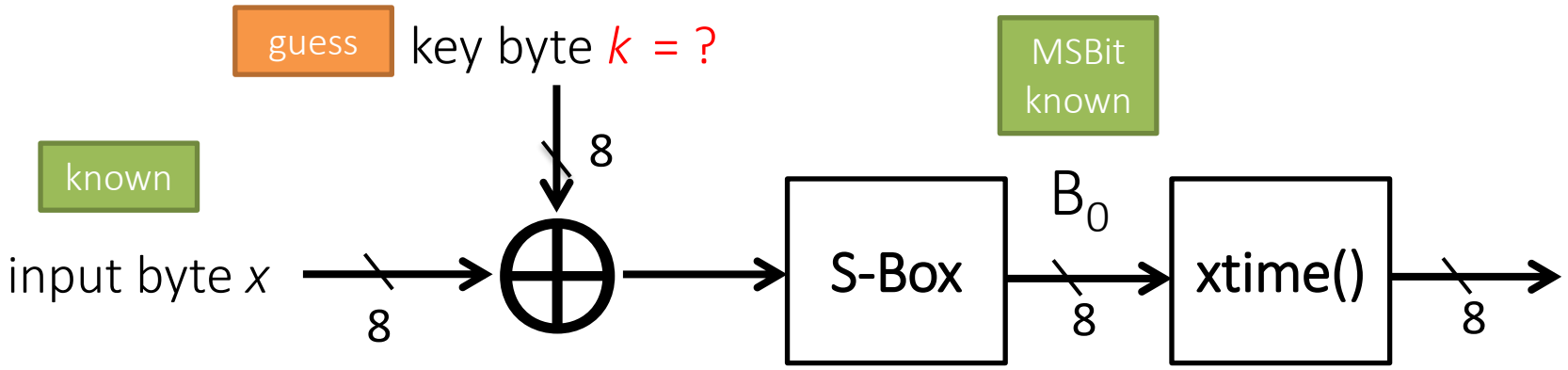
We repeat this for N different x

$x^{(0)} \ldots x^{(N-1)}$

and store the respective power traces

$p^{(0)}(t) \ldots p^{(N-1)}(t)$

Let‘s look at a single byte



1. Send *x* and “read” MSB(B_0) from power trace
2. For each candidate *k* = 00, 01, … ,FF: compute **hypothetical** MSB(B_0)

   → only 50% of candidates have correct MSB(B_0)

   → key space **halved** for **one** byte
3. Repeat with next input/trace until only one candidate left
4. Repeat for each key byte separately

Attack on MixColumns

Trace	1	2	3	4	5	6	7	8	9	10
Plaintext byte *x*	**0x00**									
MSB(B_0) observed	**0**									
#remaining keys	**128**									

key	MSB(*S(k⊕x)*)
0x00	0
0x01	0
0x02	0
0x03	0
0x04	1
0x05	0
0x06	0
0x07	1
0x08	0
0x09	0
…	
0xDE	0
…	
0xFF	0

Attack on MixColumns

Trace	1	2	3	4	5	6	7	8	9	10
Plaintext byte *x*	0x00	**0x01**								
$MSB(B_0)$ observed	0	**1**								
#remaining keys	128	**64**								

key	MSB(*S(k⊕x)*)
0x00	0
0x01	0
0x02	0
0x03	0
0x04	1
0x05	0
0x06	0
0x07	1
0x08	0
0x09	0
…	
0xDE	0
…	
0xFF	0

key	MSB(*S(k⊕x)*)
0x00	0
0x01	0
0x02	0
0x03	0
0x04	0
0x05	1
0x06	1
0x07	0
0x08	0
0x09	0
…	…
0xDE	1
…	…
0xFF	1

Attack on MixColumns

Trace	1	2	3	4	5	6	7	8	9	10
Plaintext byte *x*	0x00	0x01	**0x02**							
$MSB(B_0)$ observed	0	1	**1**							
#remaining keys	128	64	**34**							

key	MSB(*S*(*k*⊕*x*))
0x00	0
0x01	0
0x02	0
0x03	0
0x04	1
0x05	0
0x06	0
0x07	1
0x08	0
0x09	0
…	
0xDE	0
…	
0xFF	0

key	MSB(*S*(*k*⊕*x*))
0x00	0
0x01	0
0x02	0
0x03	0
0x04	0
0x05	1
0x06	1
0x07	0
0x08	0
0x09	0
…	…
0xDE	1
…	…
0xFF	1

key	MSB(*S*(*k*⊕*x*))
0x00	0
0x01	0
0x02	0
0x03	0
0x04	0
0x05	1
0x06	1
0x07	0
0x08	0
0x09	0
…	…
0xDE	1
…	…
0xFF	0

Attack on MixColumns

Trace	1	2	3	4	5	6	7	8	9	10
Plaintext byte *x*	0x00	0x01	0x02	0x03	0x04	0x05	0x06	0x07	0x08	0x09
MSB(B_0) observed	0	1	1	1	0	1	0	0	1	0
#remaining keys	128	64	34	18	9	6	3	2	2	**1**

key	MSB(*S*(*k*⊕*x*))
0x00	0
0x01	0
0x02	0
0x03	0
0x04	1
0x05	0
0x06	0
0x07	1
0x08	0
0x09	0
…	
0xDE	0
…	
0xFF	0

key	MSB(*S*(*k*⊕*x*))
0x00	0
0x01	0
0x02	0
0x03	0
0x04	0
0x05	1
0x06	1
0x07	0
0x08	0
0x09	0
…	
0xDE	1
…	
0xFF	1

key	MSB(*S*(*k*⊕*x*))
0x00	0
0x01	0
0x02	0
0x03	0
…	
0xDE	1
…	
0xFF	0

key	MSB(*S*(*k*⊕*x*)
0x00	0
0x01	0
0x02	0
0x03	0
0x04	1
0x05	1
0x06	0
0x07	1
0x08	0
0x09	0
…	
0xDE	0
…	
0xFF	0

***k* = 0xDE**

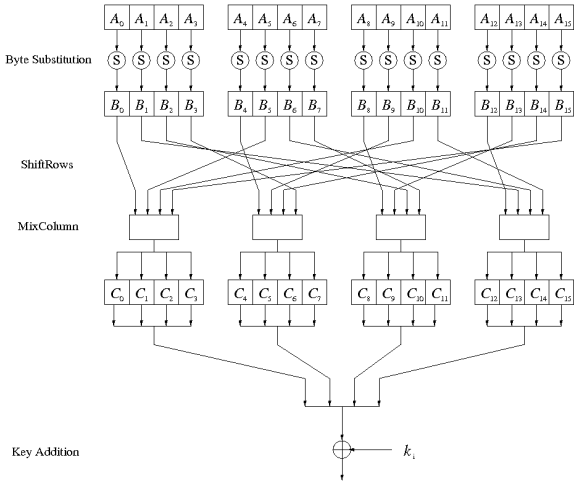
…

Protecting AES

- Attack succeeds with 8 … 10 traces – extremely efficient
- `xtime(y)` can be implemented as:

```
(( y << 1) ^ ((( y >> 7) & 1) * 0x11b))
```

- Beware optimizing compilers and CPUs with non-constant time multiplication…
- On bigger CPUs, an optimization called T-tables can be used
- We will soon see attacks that still work for the above countermeasures …



A differential timing attack on xtime()

`xtime()` timing …

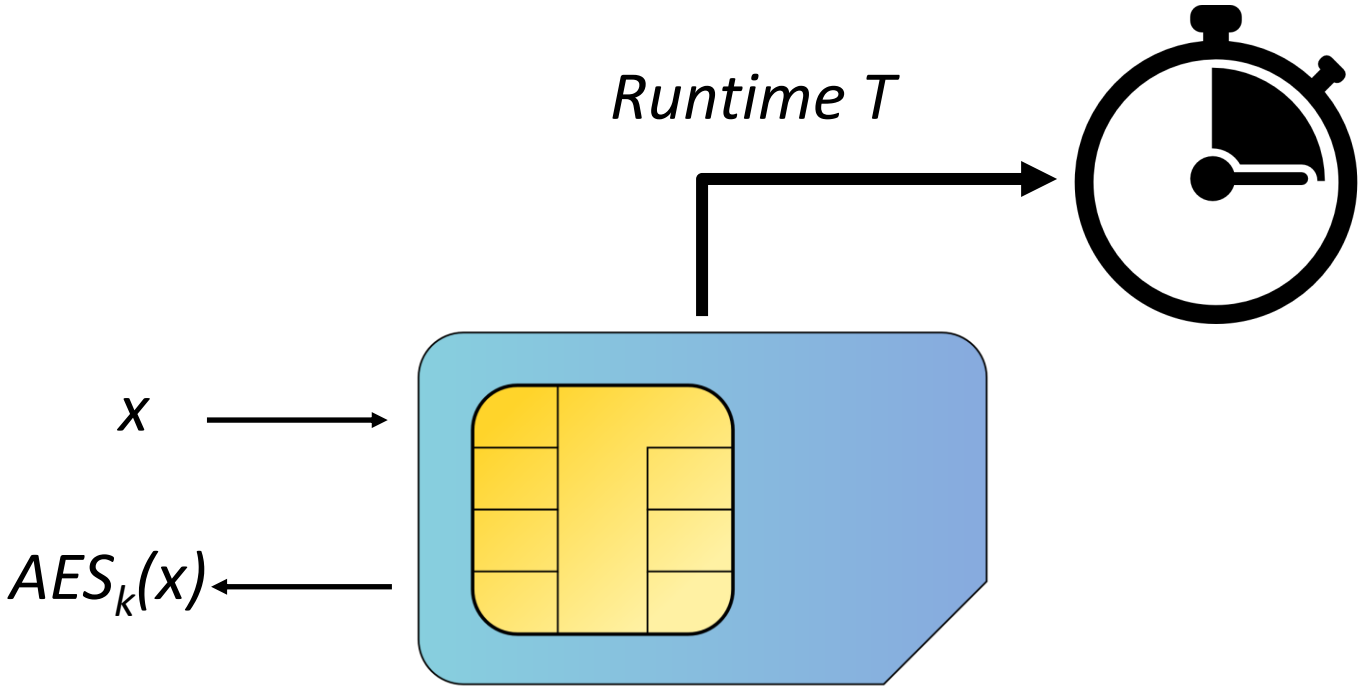
```
uint8_t tmp = B << 1;
if(MSBit(B) == 1)
   tmp ^= 0x1B;
```

- **`xtime()`** also results in non-constant execution time of the whole AES
- Assume that all other operations are constant-time - let‘s call that T_{rest}
- The complete $AES_k(x)$ has then duration

  $T(k, x) = T_{xtime}(k, x) + T_{rest}$

  where T_{xtime} is for *all* **`xtime()`**s
- If we measure the duration of the whole AES, can we **recover** k?

The setup …



We repeat this for N different x

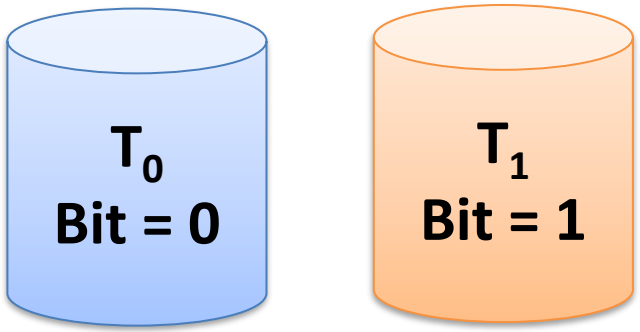
$x^{(0)} \ldots x^{(N-1)}$

and store the respective runtime

$T^{(0)} \ldots T^{(N-1)}$

The idea …

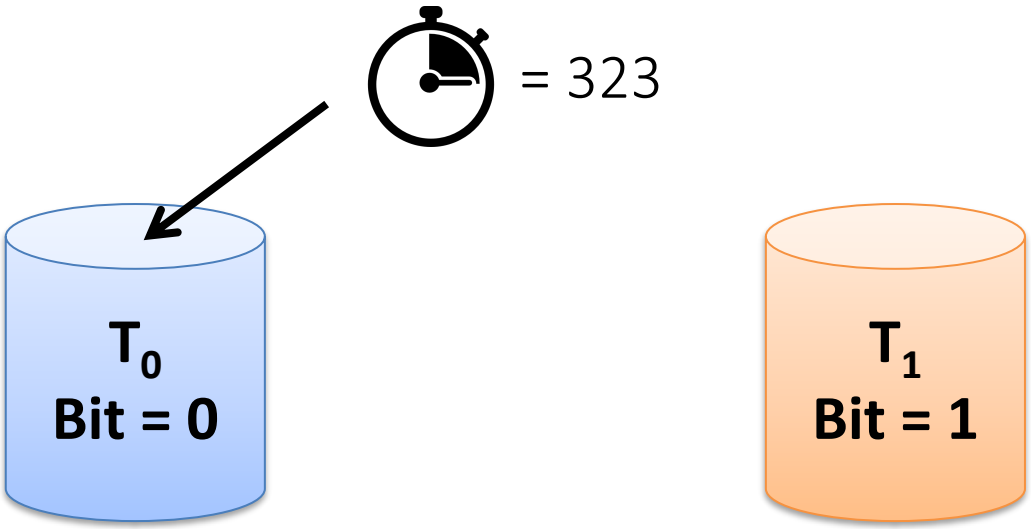
- Similar to the power analysis attack, we
  - Focus on a single key byte
  - Go over all 256 candidates for that byte
  - Go over many *(x, T)* pairs
- For each key candidate k, we compute (for each input x) if MSBit(B_0) = 1 or = 0
- We group the measured time into two "heaps"



Putting timing into two groups

Example: Guess for first key byte: k = 0x00

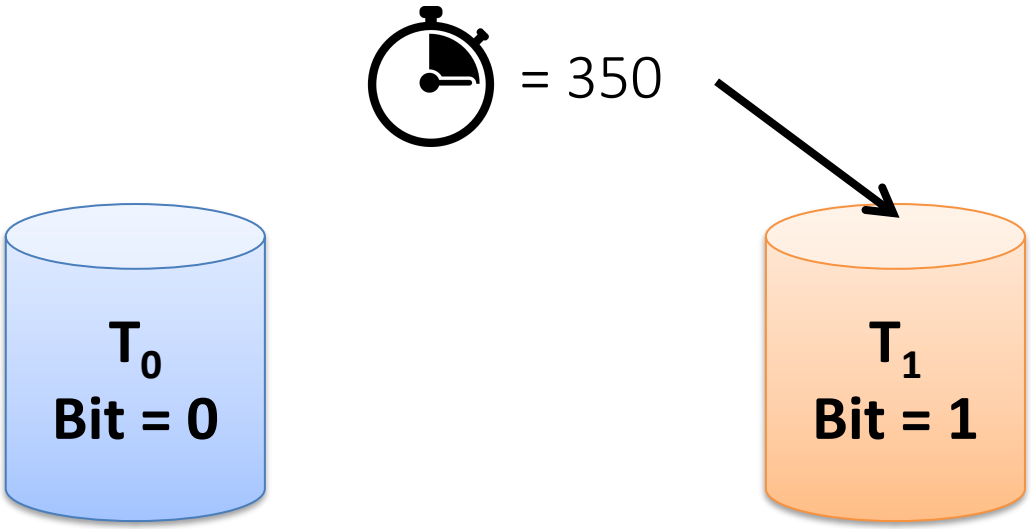
Trace	0	1	2	3	4	5	…
Plaintext x	**0x70**						…
Output S-Box B_0	**0x51**						…
MSBit(B_0)	**0**						…



Putting timing into two groups

Example: Guess for first key byte: k = 0x00

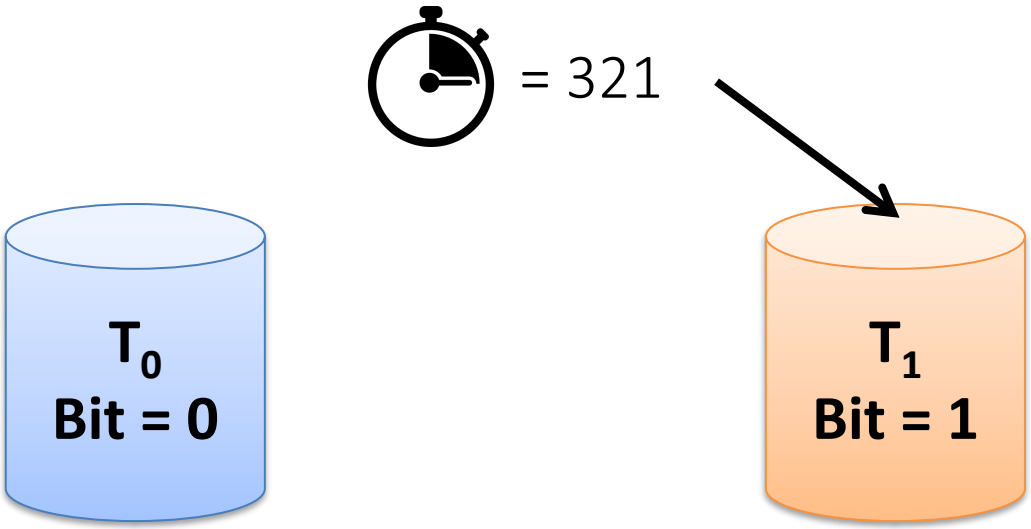
Trace	**0**	**1**	**2**	**3**	**4**	**5**	**…**
Plaintext x	0x70	**0x22**					…
Output S-Box B_0	0x51	**0x93**					…
MSBit(B_0)	0	**1**					…



Putting timing into two groups

Example: Guess for first key byte: k = 0x00

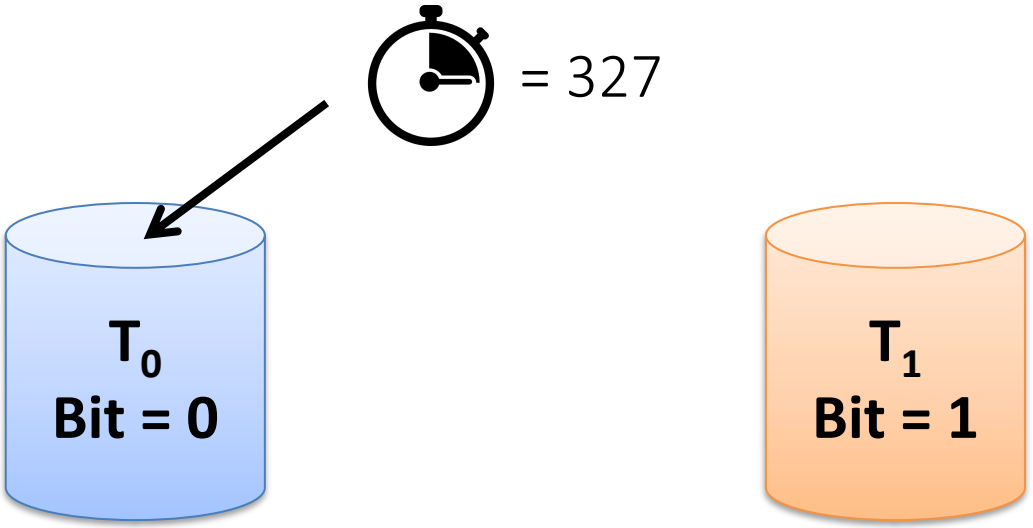
Trace	**0**	**1**	**2**	**3**	**4**	**5**	**…**
Plaintext x	0x70	0x22	**0x0C**				…
Output S-Box B_0	0x51	0x93	**0xFE**				…
MSBit(B_0)	0	1	**1**				…



Putting timing into two groups

Example: Guess for first key byte: k = 0x00

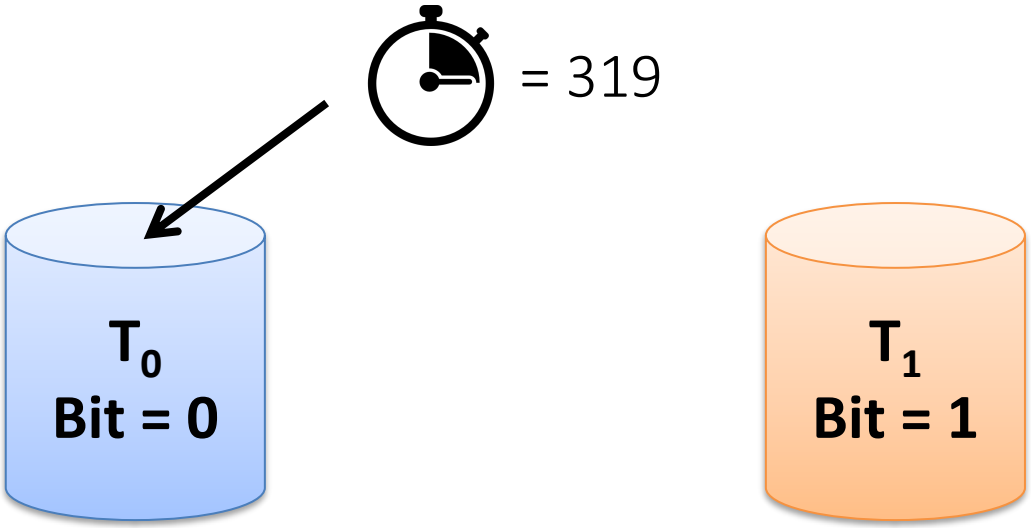
Trace	**0**	**1**	**2**	**3**	**4**	**5**	**…**
Plaintext x	0x70	0x22	0x0C	**0xFF**			…
Output S-Box B_0	0x51	0x93	0xFE	**0x16**			…
MSBit(B_0)	0	1	1	**0**			…



Putting timing into two groups

Example: Guess for first key byte: k = 0x00

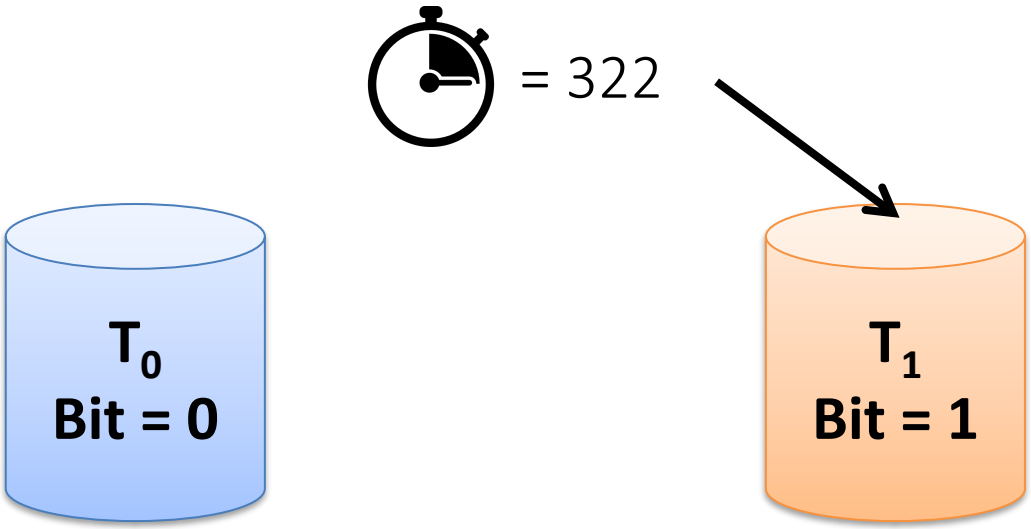
Trace	**0**	**1**	**2**	**3**	**4**	**5**	**…**
Plaintext x	0x70	0x22	0x0C	0xFF	**0x01**		…
Output S-Box B_0	0x51	0x93	0xFE	0x16	**0x7C**		…
MSBit(B_0)	0	1	1	0	**0**		…



Putting timing into two groups

Example: Guess for first key byte: k = 0x00

Trace	**0**	**1**	**2**	**3**	**4**	**5**	**…**
Plaintext x	0x70	0x22	0x0C	0xFF	0x01	**0x37**	…
Output S-Box B_0	0x51	0x93	0xFE	0x16	0x7C	**0x9A**	…
MSBit(B_0)	0	1	1	0	0	**1**	…



	Trace 0	1	2	3	4	5	6	7	…
k = 0x00	323	350	321	327	319	322	330	340	…
k = 0x01	323	350	321	327	319	322	330	340	…

	Trace 0	1	2	3	4	5	6	7	…
k = 0x00	323	350	321	327	319	322	330	340	…
k = 0x01	323	350	321	327	319	322	330	340	…

	Trace 0	1	2	3	4	5	6	7	…
k = 0x00	323	350	321	327	319	322	330	340	…
k = 0x01	323	350	321	327	319	322	330	340	…
k = 0x02	323	350	321	327	319	322	330	340	…
…	…	…	…	…	…	…	…	…	…

	Trace 0	1	2	3	4	5	6	7	…
k = 0x00	323	350	321	327	319	322	330	340	…
k = 0x01	323	350	321	327	319	322	330	340	…
k = 0x02	323	350	321	327	319	322	330	340	…
…	…	…	…	…	…	…	…	…	…

	Trace 0	1	2	3	4	5	6	7	…
k = 0x00	323	350	321	327	319	322	330	340	…
k = 0x01	323	350	321	327	319	322	330	340	…
k = 0x02	323	350	321	327	319	322	330	340	…
…	…	…	…	…	…	…	…	…	…
k = 0x12	323	350	321	327	319	322	330	340	…

	Trace 0	1	2	3	4	5	6	7	…
k = 0x00	323	350	321	327	319	322	330	340	…
k = 0x01	323	350	321	327	319	322	330	340	…
k = 0x02	323	350	321	327	319	322	330	340	…
…	…	…	…	…	…	…	…	…	…
k = 0x12	323	350	321	327	319	322	330	340	…
…	…	…	…	…	…	…	…	…	…
k = 0xFF	323	350	321	327	319	322	330	340	…

The attack …

- For each candidate *k*
  - For each *x*
    - Compute hypothetical MSbit(B_0)
    - If MSBit(B_0) == 0: put T into T_0
    - Else MSBit(B_0) == 1: put T into T_1
  - Compute averages $\overline{T}_0$ and $\overline{T}_1$ over T_0 and T_1
  - Compute $\Delta(k) = \overline{T}_1 - \overline{T}_0$
- Find max. $\Delta(k)$ → Recover one key byte k

Live-Demo

"Anything that can go wrong, will go wrong"

A timing attack on AES

Why does this work?

$$T(k, x) = T_{xtime}(k, x) + T_{rest}$$

$$\sum T_{xtime,i}(k, x) \qquad \text{const.}$$

For the correct k, we predict one term in the sum correctly, so the averages are:

$\overline{T}_0 = 0 + noise + T_{rest}$

$\overline{T}_1 = 1 + noise + T_{rest}$

$\Delta \rightarrow 1$ for many traces

For all wrong k, we predict just noise:

$\overline{T}_0 = noise + T_{rest}$

$\overline{T}_1 = noise + T_{rest}$

$\Delta \rightarrow 0$ for many traces

Take-home points

- AES – if implemented naively – is very vulnerable to side-channel attacks
- Power leakage reveals the key in few traces
- Timing leakage reveals the key with differential attack
- **Next lecture:** combining these two ideas into **Differential Power Analysis**

Thanks!

Questions / discussion?

d.f.oswald@bham.ac.uk